RMI VS CORBA

- Presented by :
- Jitendra Raut
- Komal Basnet
- Asha Giri

TABLE OF CONTENT :

- I. CONCEPT AND PURPOSE.
- II. PLATFORM DEPENDENCE.
- III. COMMUNICATION MODEL..
- IV. COMPLEXITY.
- V. PERFORMANCE.
- VI. EVOLUTION AND CURRENT STATUS.
- VII. ADVANTAGES AND DISADVANTAGES.
- VIII. SUMMARY TABLE.

I. CONCEPT AND PURPOSE.

- **RMI (Remote Method Invocation)**:
- RMI stands for Remote Method Invocation.
- It is a mechanism that allows an object residing in one system (JVM) to access/invoke an object running on another JVM.
- RMI is used to build distributed applications; it provides remote communication between Java programs.

•**CORBA (Common Object Request Broker Architecture)**:CORBA is a language-independent specification that allows objects written in different programming languages to communicate with each other.
•It provides a framework for making remote procedure calls (RPCs) between objects located on different machines, regardless of the programming language.
•CORBA allows interoperability between different systems and platforms (e.g., Java, C++, Python, etc.).

II. PLATFORM DEPENDENCE.

Platform Dependence

RMI:

- RMI is platform-dependent and works only on systems that have Java Runtime Environments (JRE).
- It is more suitable for applications that are primarily written in Java and run in Java environments.

• CORBA:

- CORBA is designed to be platform-independent, meaning it can be used on systems running different operating systems (Windows, Linux, etc.), as long as they support CORBA.
- It offers greater flexibility for heterogeneous environments.

III. COMMUNICATION MODEL..

- **RMI**:
- RMI follows a client-server model where a client can invoke methods on a remote server object.
- RMI works on a single object-level communication, meaning one object invokes methods on another object.

- **CORBA**:
- CORBA uses an Object Request Broker (ORB) to enable communication between distributed objects, which acts as an intermediary to facilitate the interaction between objects.
- CORBA can handle more complex communication patterns such as multiple clients invoking different methods on various remote objects.

IV. COMPLEXITY.

RMI:

•RMI is relatively simpler to use, especially for Java developers, since it is part of the Java platform.
•Setting up and using RMI typically requires less configuration.

CORBA:

•CORBA is more complex to implement and use because of its cross-language nature and additional features like interface definition languages (IDL), object marshaling, and ORBs.
•It requires the creation of an interface definition using IDL (Interface Definition Language), which must be compiled to generate language-specific stubs and skeletons.

V. PERFORMANCE.

RMI:

- RMI is relatively faster in Java environments because it is optimized for Java objects and does not require additional translation between languages.
- Since it is Java-specific, there is less overhead when transmitting data and invoking methods.

CORBA:

- CORBA may introduce more overhead due to its generality and the need for the ORB to handle communication between different language systems.
- The performance can vary depending on the complexity of the system and the communication between heterogeneous systems.

VI. EVOLUTION AND CURRENT STATUS.

•**RMI**:
•RMI is still widely used in Java-based systems but has been largely supplanted by more modern alternatives such as Web Services, REST, and GRPC.
•RMI may not be ideal for new projects unless they are specifically Java-centric.

- **CORBA**:
- CORBA's popularity has declined in recent years due to the emergence of newer, simpler, and more efficient technologies like web services (SOAP, REST) and messaging protocols (AMQP, MQTT).
- It is still used in legacy systems, especially in industries such as aerospace, telecommunications, and finance.

VII. ADVANTAGES AND DISADVANTAGES.

Advantages:

RMI

- Simpler to implement in a Java-only environment, faster performance within Java ecosystems.

CORBA

- Platform and language independent, offers greater flexibility in heterogeneous environments.

Disadvantages:

RMI

- Limited to Java, not suitable for applications that need to support multiple languages or platforms.

CORBA

- More complex and may involve higher overhead for smaller-scale applications.

VIII. SUMMARY TABLE.

Feature	RMI	CORBA
Language Support	Java-only	Multi-language (Java, C++, Python, etc.)
Interoperability	Java-to-Java communication	Cross-language and cross-platform
Platform	Java-based platforms (JRE)	Multi-platform (Windows, Linux, etc.)
Communication Model	Simple client-server model	Complex communication via ORB
Complexity	Relatively simple	More complex due to IDL and ORB
Performance	Faster for Java applications	May introduce overhead
Current Status	Less popular for new projects	Declining, mostly in legacy systems

THANK YOU!!!!